



BASE DE DATOS AVANZADAS Y BIG DATA

UPN.EDU.PE

Semana 02



Control de Excepciones.

Procedimientos Almacenados con Flujos Condicionales y Vistas.



Presentación de la sesión

Logro de sesión

Al término de la sesión los estudiantes podrán controlar excepciones, crear vistas y procedimientos almacenados con flujos para la aplicación de la lógica de procesos dentro de la base de datos.

Temario

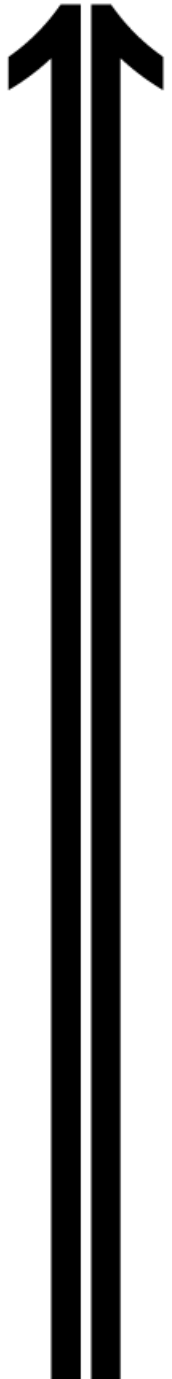
- Control de Excepciones.
- Procedimientos almacenados con flujos condicionales y vistas



¿Qué es un control de excepciones?

¿Qué es un procedimiento almacenado?

¿Qué es una vista?



CONTROL DE EXCEPCIONES

GESTIÓN DE ERRORES EN SQL SERVER



La gestión de errores en SQL Server nos da **control sobre el código Transact-SQL**. El manejo de errores de SQL Server puede ser tan simple como simplemente registrar que algo sucedió, o podríamos ser nosotros tratando de corregir un error. Incluso puede traducir el error en lenguaje SQL porque todos sabemos cómo los mensajes de error técnicos de SQL Server pueden volverse sin sentido y difíciles de entender. Afortunadamente, tenemos la oportunidad de **traducir esos mensajes** en algo más significativo para transmitir a los usuarios, desarrolladores, etc.

```
SQLQuery1.sql - JIT...Works2012 (sa (52))* ×
1  --Here we will get statement Abortion
2  PRINT 'Beginning of execution'
3  INSERT INTO Book(BookID, Name , Price)
4  VALUES(1, 'Demo Book1', 100)
5  INSERT INTO Book(BookID, Name , Price)
6  VALUES(1, 'Demo Book2', 200)
7  PRINT 'End of execution'
```

100 %

Messages

Beginning of execution

(1 row(s) affected)
Msg 2627, Level 14, State 1, Line 5
Violation of PRIMARY KEY constraint 'PK__Book__3DE0C227EA4D3793'.
Cannot insert duplicate key in object 'dbo.Book'. The duplicate key value is (1, 'Demo Book2').
The statement has been terminated.
End of execution



TRY...CATCH (TRANSACT-SQL)



Implementa un mecanismo de control de errores para Transact-SQL que es similar al control de excepciones en los lenguajes Microsoft Visual C# y Microsoft Visual C++. Se puede incluir un grupo de instrucciones Transact-SQL en un bloque **TRY**. Si se produce un error en el bloque TRY, el control se suele transferir a otro grupo de instrucciones que está incluido en un bloque **CATCH**

```
BEGIN TRY
    { sql_statement | statement_block }
END TRY
BEGIN CATCH
    [ { sql_statement | statement_block } ]
END CATCH
[ ; ]
```

TRY...CATCH (TRANSACT-SQL)



```
select 1/0
```

100 %

Results Messages

Msg 8134, Level 16, State 1, Line 15
Divide by zero error encountered.

Completion time: 2023-02-20T22:25:41.1040790-05:00

```
BEGIN TRY
    SELECT 1/0
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS ErrorNumber,
        ERROR_SEVERITY() AS ErrorSeverity,
        ERROR_STATE() AS ErrorState,
        ERROR_PROCEDURE() AS ErrorProcedure,
        ERROR_LINE() AS ErrorLine,
        ERROR_MESSAGE() AS ErrorMessage
END CATCH
GO
```

```
BEGIN TRY
    SELECT 1/0
END TRY
BEGIN CATCH
    SELECT
        ERROR_NUMBER() AS ErrorNumber,
        ERROR_SEVERITY() AS ErrorSeverity,
        ERROR_STATE() AS ErrorState,
        ERROR_PROCEDURE() AS ErrorProcedure,
        ERROR_LINE() AS ErrorLine,
        ERROR_MESSAGE() AS ErrorMessage
END CATCH
GO
```

100 %

Results Messages

(No column name)

	ErrorNumber	ErrorSeverity	ErrorState	ErrorProcedure	ErrorLine	ErrorMessage
1	8134	16	1	NULL	2	Divide by zero error encountered.

RECUPERAR INFORMACIÓN SOBRE ERRORES



En el ámbito de un bloque **CATCH**, se pueden utilizar las siguientes funciones del sistema:

ERROR_NUMBER() devuelve el número del error.

ERROR_SEVERITY() devuelve la gravedad.

ERROR_STATE() devuelve el número de estado del error.

ERROR_PROCEDURE() devuelve el nombre del procedimiento almacenado o desencadenador donde se produjo el error.

ERROR_LINE() devuelve el número de línea de la rutina que provocó el error.

ERROR_MESSAGE() devuelve el texto completo del mensaje de error. El texto incluye los valores proporcionados para los parámetros sustituibles, como las longitudes, nombres de objeto o tiempos.

*Estas funciones devuelven **NULL** si se las llama desde fuera del ámbito del bloque CATCH. Con ellas se puede recuperar información sobre los errores desde cualquier lugar dentro del ámbito del bloque CATCH*

TRY...CATCH (TRANSACTION-SQL)



Si tratamos de eliminar un registro Cliente que está siendo usado en la tabla Pedidos, nos va a provocar un error de Integridad Referencial.

```
use Negocios
go

/*Eliminar un registro que tenga referencia*/
DELETE FROM Ventas.clientes
WHERE IdCliente = 'ALFKI'
GO
```

Messages

Msg 547, Level 16, State 0, Line 20

The DELETE statement conflicted with the REFERENCE constraint "FK__pedidosca__IdCli__398D8EEE". The conflict occurred in database "
The statement has been terminated.

Completion time: 2023-02-20T22:53:40.2589629-05:00

100 %

⚠ Query completed with errors.

DESKTOP-OGUAPD\MSSQLSERVER...

DESKTOP-OGUAPD\VICTOR...

Negocios

00:00:00

0 rows

TRY...CATCH (TRANSACT-SQL)



La misma sentencia pero controlando el error.

```
BEGIN TRY
  DELETE FROM Ventas.clientes
  WHERE IdCliente = 'ALFKI'
END TRY
BEGIN CATCH
  PRINT 'OCURRIÓ UN ERROR'
END CATCH
GO
```

Messages

(0 rows affected)
OCURRIÓ UN ERROR

Completion time: 2023-02-20T23:03:47.9591465-05:00

TRY...CATCH (TRANSACT-SQL)



Podemos precisar el error y ser específico con la excepción haciendo uso de la variable global **@@ERROR** el cual nos proporciona el número de error.

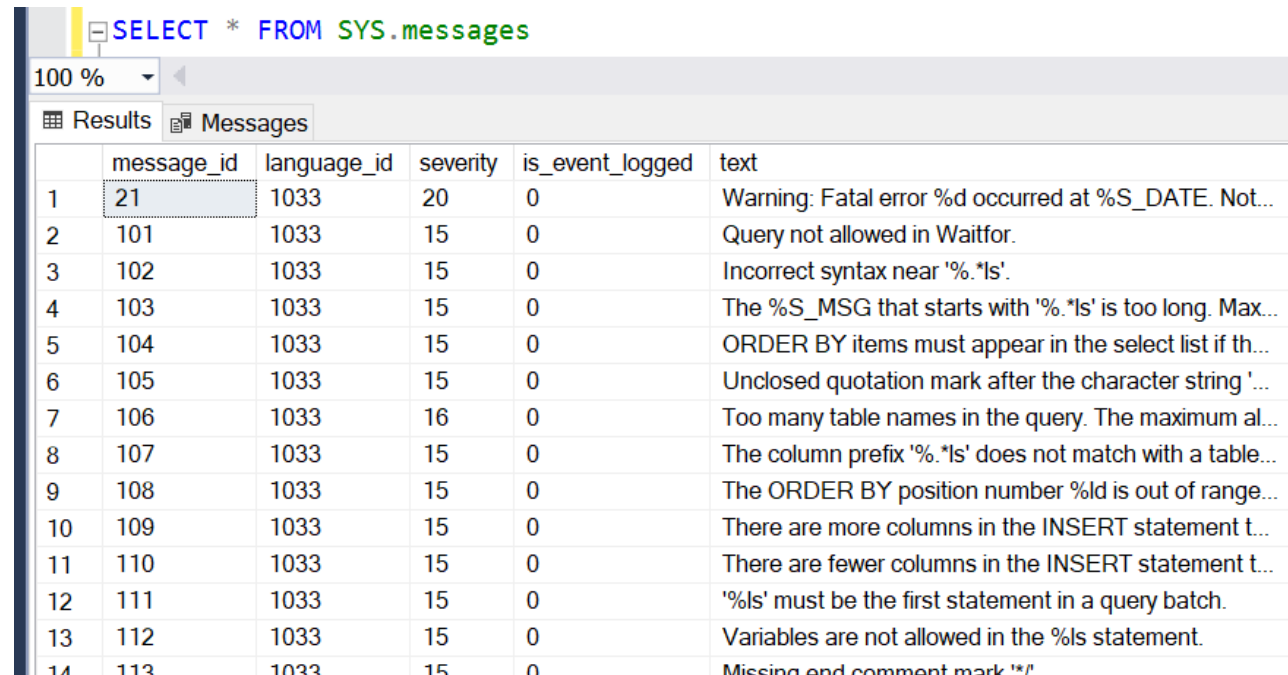
Todos los mensaje, incluidos los de error los podemos observar desde **SYS.messages**

Por ejemplo:

2627 Primary Key duplicado

547 conflicto de Constraint, pe FK

8134 División entre cero



The screenshot shows a SQL query window with the text `SELECT * FROM SYS.messages`. Below the query, there are tabs for 'Results' and 'Messages'. The 'Messages' tab is selected, displaying a table of system messages. The table has columns: message_id, language_id, severity, is_event_logged, and text. The first row (message_id 21) is highlighted.

	message_id	language_id	severity	is_event_logged	text
1	21	1033	20	0	Warning: Fatal error %d occurred at %S_DATE. Not...
2	101	1033	15	0	Query not allowed in Waitfor.
3	102	1033	15	0	Incorrect syntax near '%. *Is'.
4	103	1033	15	0	The %S_MSG that starts with '%. *Is' is too long. Max...
5	104	1033	15	0	ORDER BY items must appear in the select list if th...
6	105	1033	15	0	Unclosed quotation mark after the character string '...
7	106	1033	16	0	Too many table names in the query. The maximum al...
8	107	1033	15	0	The column prefix '%. *Is' does not match with a table...
9	108	1033	15	0	The ORDER BY position number %ld is out of range...
10	109	1033	15	0	There are more columns in the INSERT statement t...
11	110	1033	15	0	There are fewer columns in the INSERT statement t...
12	111	1033	15	0	'%Is' must be the first statement in a query batch.
13	112	1033	15	0	Variables are not allowed in the %Is statement.
14	113	1033	15	0	Missing end comment mark '*/'

TRY...CATCH (TRANSACT-SQL)



Validamos el número de error con @@ERROR.

```
BEGIN TRY
  DELETE FROM Ventas.clientes
  WHERE IdCliente = 'ALFKI'
END TRY
BEGIN CATCH
  IF @@ERROR = 547
    PRINT 'OCURRIÓ UN ERROR DE INTEGRIDAD REFERENCIAL'
END CATCH
GO
```

Messages

(0 rows affected)

OCURRIÓ UN ERROR DE INTEGRIDAD REFERENCIAL

Completion time: 2023-02-20T23:33:27.9409270-05:00

RAISERROR



Se usa para devolver mensajes definidos por el usuario a las aplicaciones con el mismo formato de un error del sistema o de un mensaje de advertencia generado por el motor de base de datos SQL Server

SINTAXIS

RAISERROR(MENSAJE, severity, state)

Dependiendo de la severidad se tomará como Informativo, Warning o Error

A partir de la severidad 11, RAISERROR saltará al bloque CATCH

TABLA 1: RAISERROR Severity Categories

Severity Range	Error Number Info	@@ERROR	Logged As
1 hasta 9	In black	NOT SET	Informational
10	Not provided	NOT SET	Informational
11 hasta 14	In red	SET	Informational
15	In red	SET	Warning
16	In red	SET	Error

TRY...CATCH (TRANSACT-SQL)



Verificamos el **Raiserror**, cambiando la severidad

```
DECLARE @NUM INT= 10
BEGIN TRY
  IF @NUM < 11
    RAISERROR('ERROR GENERADO',10,1)

  PRINT 'PASA POR ACÁ'

END TRY
BEGIN CATCH

  PRINT '!!ENTRA AL BLOQUE CATCH'
  PRINT @@ERROR
  PRINT ERROR_NUMBER()
  PRINT ERROR_STATE()
  PRINT ERROR_SEVERITY()
  PRINT ERROR_LINE()
  PRINT ERROR_PROCEDURE()
  PRINT ERROR_MESSAGE()
END CATCH
GO
```

```
DECLARE @NUM INT= 10
BEGIN TRY
  IF @NUM < 11
    RAISERROR('ERROR GENERADO',10,1)

  PRINT 'PASA POR ACÁ'

END TRY
BEGIN CATCH
  PRINT '!!ENTRA AL BLOQUE CATCH'
  PRINT @@ERROR
  PRINT ERROR_NUMBER()
  PRINT ERROR_STATE()
  PRINT ERROR_SEVERITY()
  PRINT ERROR_LINE()
  PRINT ERROR_PROCEDURE()
  PRINT ERROR_MESSAGE()
END CATCH
GO
```

100 %

Messages

ERROR GENERADO
PASA POR ACÁ

Completion time: 2023-02-21T00:13:52.6957461-05:00

```
DECLARE @NUM INT= 10
BEGIN TRY
  IF @NUM < 11
    RAISERROR('ERROR GENERADO',16,1)

  PRINT 'PASA POR ACÁ'

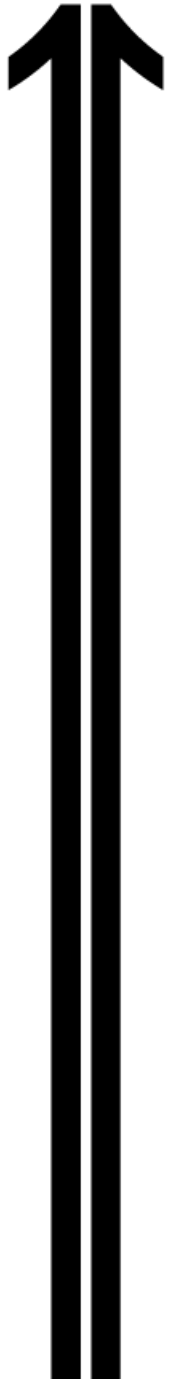
END TRY
BEGIN CATCH
  PRINT '!!ENTRA AL BLOQUE CATCH'
  PRINT @@ERROR
  PRINT ERROR_NUMBER()
  PRINT ERROR_STATE()
  PRINT ERROR_SEVERITY()
  PRINT ERROR_LINE()
  PRINT ERROR_PROCEDURE()
  PRINT ERROR_MESSAGE()
END CATCH
GO
```

100 %

Messages

!!ENTRA AL BLOQUE CATCH
0
50000
1
16
4
ERROR GENERADO

Completion time: 2023-02-21T00:14:24.1161934-05:00



PROCEDIMIENTOS ALMACENADOS

PROCEDIMIENTOS ALMACENADOS



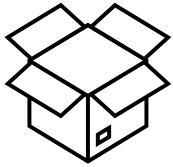
Es un **conjunto de instrucciones SQL** predefinidas que se **almacenan en el servidor** de base de datos y se pueden llamar y ejecutar en cualquier momento mediante una llamada desde una aplicación o un cliente de base de datos.

Un procedimiento almacenado está compuesto por una o varias instrucciones SQL que pueden ser ejecutadas de forma secuencial, y **puede aceptar parámetros** de entrada y devolver resultados de salida.

```
SQLQuery13.sql - S...\PC\Shreeharsh (53)) X
USE [SqlHowTo]
GO
/***** Object:  StoredProcedure [dbo].[InsertPersonalDetails]
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
-- =====
-- Author:      <Author,,Name>
-- Create date: <Create Date,,>
-- Description: <Description,,>
-- =====
ALTER PROCEDURE [dbo].[InsertPersonalDetails]
-- Add the parameters for the stored procedure here
    @FirstName varchar(50),
    @LastName varchar(50),
    @Age smallint,
    @Active bit
AS
BEGIN
-- SET NOCOUNT ON added to prevent extra result sets from
-- interfering with SELECT statements.
SET NOCOUNT ON;

-- Insert statements for procedure here
INSERT INTO PersonalDetails
    (FirstName, LastName, Age, Active)
VALUES
    (@FirstName, @LastName, @Age, @Active)
END
```

PROCEDIMIENTOS ALMACENADOS

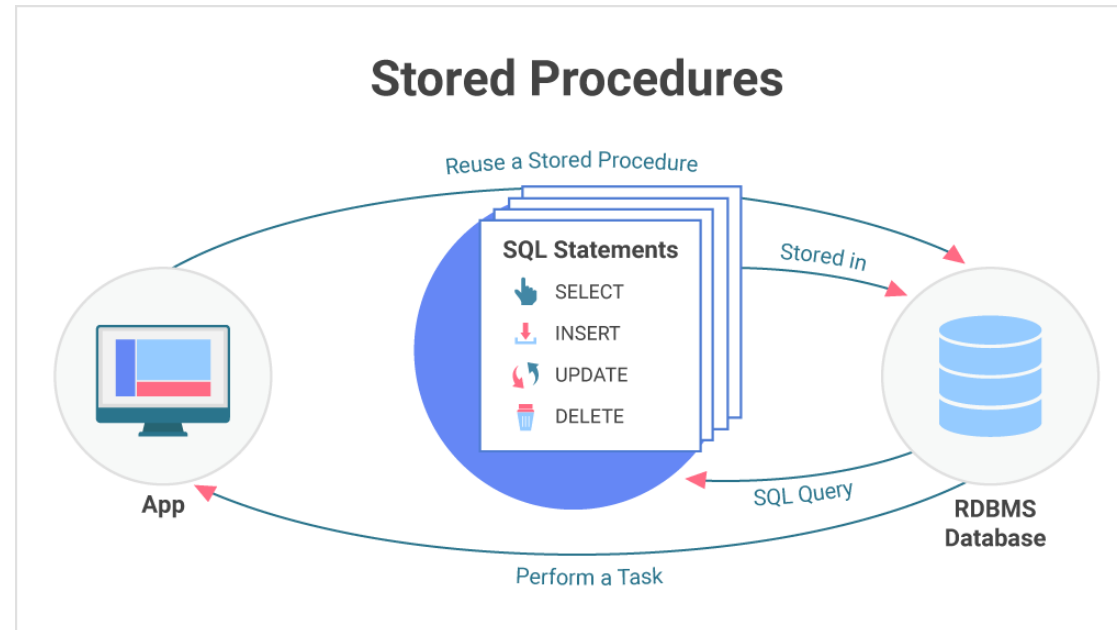


Modularidad

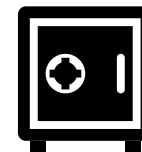
Algunas de las ventajas



Rendimiento



Simplificación del código



Seguridad

PROCEDIMIENTOS ALMACENADOS



Los procedimientos almacenados pueden:

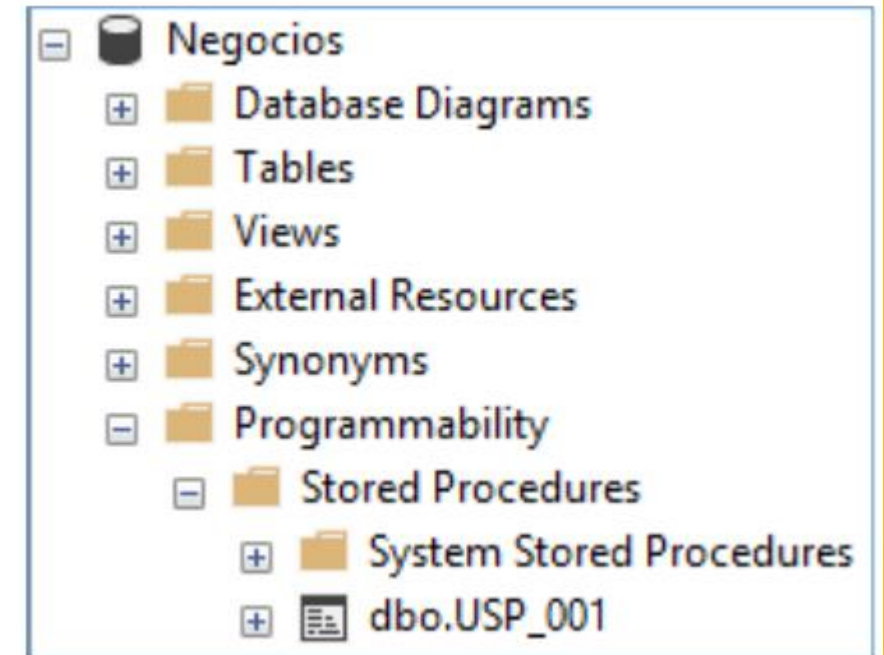
- ✓ Incluir parámetros.

- ✓ Llamar a otros procedimientos.

- ✓ Devolver un valor de estado a un procedimiento de llamada o lote para indicar el éxito o el fracaso del mismo y la razón de dicho fallo.

- ✓ Devolver valores de parámetros a un procedimiento de llamada o lote.

- ✓ Ejecutarse en SQL Server remotos



PROCEDIMIENTOS ALMACENADOS



Para visualizar los procedimientos en la BD

```
SELECT * FROM sys.procedures
```

SELECT * FROM sys.procedures								
100 %								
Results Messages								
	name	object_id	principal_id	schema_id	parent_object_id	type	type_desc	create_date
1	uspGetBillOfMaterials	887674210	NULL	1	0	P	SQL_STORED_PROCEDURE	2017-10-27 14:33:15.
2	uspGetEmployeeManagers	903674267	NULL	1	0	P	SQL_STORED_PROCEDURE	2017-10-27 14:33:15.
3	uspGetManagerEmployees	919674324	NULL	1	0	P	SQL_STORED_PROCEDURE	2017-10-27 14:33:15.
4	uspGetWhereUsedProductID	935674381	NULL	1	0	P	SQL_STORED_PROCEDURE	2017-10-27 14:33:15.
5	uspUpdateEmployeeHireInfo	951674438	NULL	5	0	P	SQL_STORED_PROCEDURE	2017-10-27 14:33:15.
6	uspUpdateEmployeeLogin	967674495	NULL	5	0	P	SQL_STORED_PROCEDURE	2017-10-27 14:33:15.
7	uspPrintError	981578535	NULL	1	0	P	SQL_STORED_PROCEDURE	2017-10-27 14:33:01.
8	uspUpdateEmployeePersonalInfo	983674552	NULL	5	0	P	SQL_STORED_PROCEDURE	2017-10-27 14:33:15.
9	uspLogError	997578592	NULL	1	0	P	SQL_STORED_PROCEDURE	2017-10-27 14:33:01.
10	uspSearchCandidateResumes	999674609	NULL	1	0	P	SQL_STORED_PROCEDURE	2017-10-27 14:33:15.

PROCEDIMIENTOS ALMACENADOS



Sintaxis

CREACIÓN

```
CREATE PROCEDURE nombre_del_procedimiento
    @parametro1 tipo_de_dato,
    @parametro2 tipo_de_dato,
    ...
AS
BEGIN
    -- Instrucciones SQL aquí
END
```

MODIFICACIÓN

```
ALTER PROCEDURE nombre_del_procedimiento
    @parametro1 tipo_de_dato,
    @parametro2 tipo_de_dato,
    ...
AS
BEGIN
    -- Instrucciones SQL aquí
END
```

ELIMINACIÓN

```
DROP PROCEDURE nombre_del_procedimiento
```

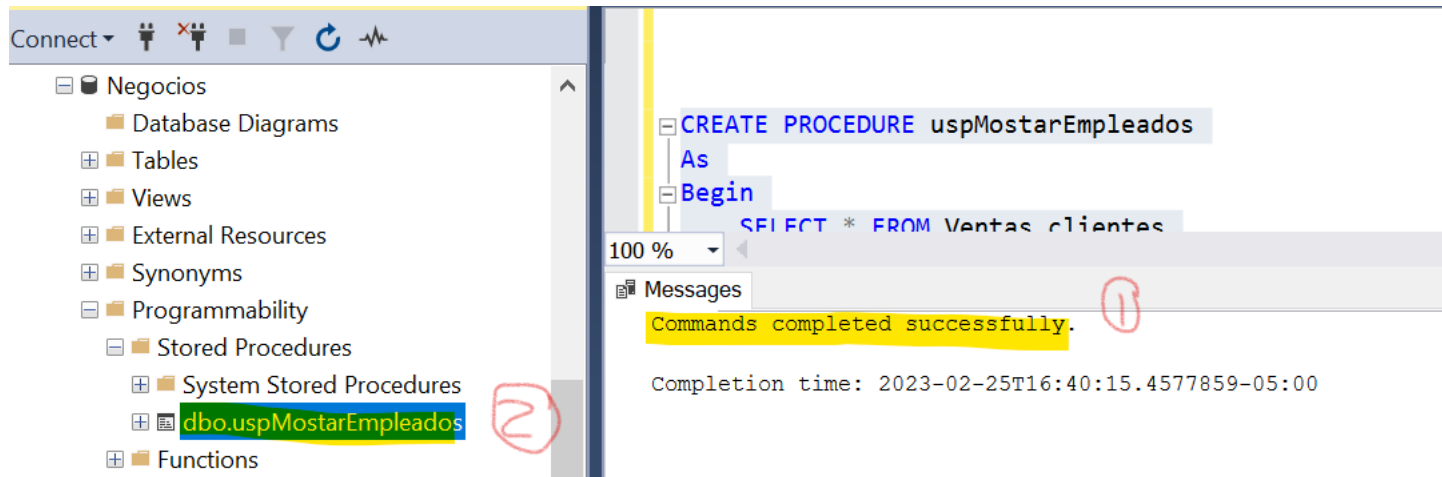
PROCEDIMIENTOS ALMACENADOS



Ejemplo1

Crear un procedimiento almacenado en la BD
Negocios que muestre la lista de todos los clientes

```
CREATE PROCEDURE uspMostarEmpleados
As
Begin
    SELECT * FROM Ventas.clientes
End
GO
```



PROCEDIMIENTOS ALMACENADOS



Para ejecutar un procedimiento almacenado

EXECUTE nombreProcedimiento

En nuestro procedimiento creado

EXECUTE uspMostarEmpleados



EXECUTE uspMostarEmpleados

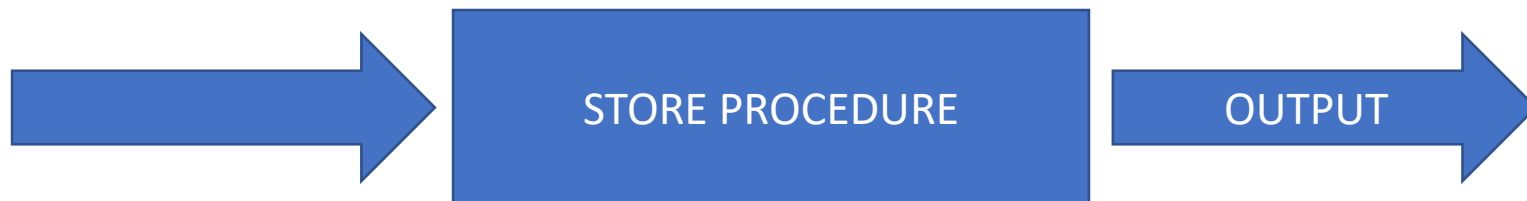
	IdCliente	NomCliente	DirCliente	idpais	fonoCliente
1	ALFKI	Alfreds Futterkiste	Obere Str. 57	002	030-0074321
2	ANATR	Ana Trujillo Emparedados y helados	Avda. de la Constitucion 2222	005	(5) 555-4729
3	ANTON	Antonio Moreno Taqueria	Mataderos 2312	007	(5) 555-3932
4	AROUT	Around the Horn	120 Hanover Sq.	004	(71) 555-7788
5	BERGS	Berglunds snabbköp	Berguvsvägen 8	006	0921-12 34 65
6	BLAUS	Blauer See Delikatessen	Forsterstr. 57	001	0621-08460
7	BLONP	Blondel père et fils	24, place Kleber Estrasburgo	008	88.60.15.31
8	BOLID	Bolido Comidas preparadas	C/ Araquil, 67	009	(91) 555 91 99
9	BONAP	Bon app	12, rue des Bouchers	001	91.24.45.41
10	BOTTM	Bottom-Dollar Markets	23 Tsawassen Blvd.	003	(604) 555-3745
11	BSBEV	B's Beverages	Fauntleroy Circus	009	(71) 555-1212
12	CACTU	Cactus Comidas para llevar	Cerrito 333	002	(1) 135-4892
13	CENTC	Centro comercial Moctezuma	Sierras de Granada 9993	008	(5) 555-7293

PROCEDIMIENTOS ALMACENADOS



Especificar parámetros

- Especificar un nombre: Deben empezar por un solo carácter @, como una variable estándar de TRANSACT-SQL.
- Especificar una dirección: Puede ser de **entrada**, o de **salida**; por defecto es un parámetro de entrada. Para especificar un parámetro de salida, debe indicar la palabra clave **OUTPUT** en la definición del parámetro del procedimiento almacenado.
- Especificar un valor predeterminado: Puede crearse parámetros especificando un valor predeterminado para los mismos. Al ejecutar el procedure, se utilizará el valor predeterminado si no se ha especificado ningún otro.



PROCEDIMIENTOS ALMACENADOS



Ejemplo2

Modificar el procedimiento **uspMostrarEmpleados** para que reciba un parámetro de entrada para el **idPais** y este sirva de filtro en la consulta

```
ALTER PROCEDURE uspMostrarEmpleados
    @p_idPais char(3)
As
Begin
    SELECT * FROM Ventas.clientes
    WHERE idpais = @p_idPais
End
GO
```

100 %

Messages

Commands completed successfully.

Completion time: 2023-02-25T17:42:56.4184028-05:00

```
ALTER PROCEDURE uspMostrarEmpleados
    @p_idPais char(3)
As
Begin

    SELECT * FROM Ventas.clientes
    WHERE idpais = @p_idPais
End
GO
```

PROCEDIMIENTOS ALMACENADOS



Para ejecutar un procedimiento modificado con parametro

```
EXECUTE uspMostarEmpleados @p_idPais = '002'
```

```
-- 0
```

```
EXECUTE uspMostarEmpleados '002'
```



The screenshot shows a SQL Server Enterprise Manager interface. At the top, a query window displays the following code:

```
EXECUTE uspMostarEmpleados @p_idPais = '002'  
-- 0  
EXECUTE uspMostarEmpleados '002'
```

Below the query window, the 'Results' tab is selected, showing a table with 9 rows and 6 columns. The columns are: IdCliente, NomCliente, DirCliente, idpais, and fonoCliente. The first row is highlighted.

	IdCliente	NomCliente	DirCliente	idpais	fonoCliente
1	ALFKI	Alfreds Futterkiste	Obere Str. 57	002	030-0074321
2	CACTU	Cactus Comidas para llevar	Cerrito 333	002	(1) 135-4892
3	GROSR	GROSELLA-Restaurante	5ª Ave. Los Palos Grandes	002	(2) 283-3397
4	LAMAI	La maison de Asie	1 rue Alsace-Lorraine	002	61.77.61.11
5	LINOD	LINO-Delicateses	Ave. 5 de Mayo Porlamar	002	(8) 34-93-93
6	PICCO	Piccolo und mehr	Geislweg 14	002	6562-9723
7	SANTG	Sante Gourmet	Erling Skakkas gate 78	002	07-98 92 47
8	WARTH	Wartian Herkku	Torikatu 38	002	981-443655
9	WOLZA	Wolski Zajazd	ul. Filtrowa 68	002	(26) 642-7012

PROCEDIMIENTOS ALMACENADOS



Ejemplo3

Crear el procedimiento uspCalculaSuma, que reciba dos parámetros numéricos y devuelva el resultado con un parámetro de salida **OUTPUT**

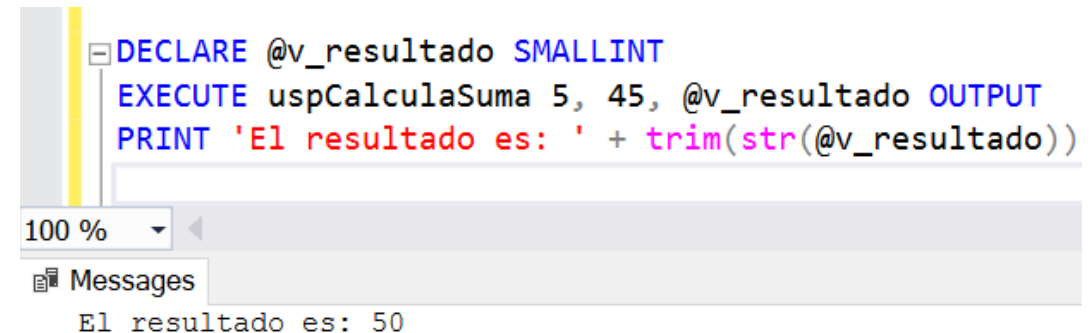
```
CREATE PROCEDURE uspCalculaSuma
    @p_num1 smallint, @p_num2 smallint,
    @p_resultado smallint OUTPUT
AS
BEGIN

    SET @p_resultado = @p_num1 + @p_num2

END
```

Ejecutando

```
DECLARE @v_resultado SMALLINT
EXECUTE uspCalculaSuma 5, 45,
    @v_resultado OUTPUT
PRINT 'El resultado es: ' +
    trim(str(@v_resultado))
```





INTEGRADOR

Crear un procedimiento en el esquema RRHH que permita insertar un nuevo Empleado, se debe tener en consideración lo siguiente:

- ✓ El ID del empleado debe calcularse en el procedimiento
- ✓ Se debe validar la existencia del idDistrito para asegurar la integridad referencial
- ✓ Se debe controlar el error en la sentencia de inserción TRY CATCH
- ✓ Todos los mensajes de resultados se deben devolver con un parámetro de salida
OUTPUT

PROCEDIMIENTOS ALMACENADOS



```
CREATE PROCEDURE Compras.uspInsertaEmpleado
@p_ApeEmp varchar(50),
@p_NomEmp varchar(50),
@p_FecNac datetime,
@p_DirEmp varchar(60),
@p_idDist int,
@p_fonoEmp varchar(15),
@p_idCargo int,
@p_FecContrata datetime,
@P_resultado varchar(500) OUTPUT
AS
Begin
declare @v_idEmp int

--Validamos la existencia del idDistrito porintegridad referencial
IF EXISTS(SELECT * FROM RRHH.Distritos WHERE idDistrito = @p_idDist)
BEGIN
--Calculamos el nuevo Id de empleado
SELECT @v_idEmp = MAX(IdEmpleado) + 1FROM RRHH.empleados
--controlamos el error en la inserción
BEGIN TRY
INSERT INTO RRHH.empleados VALUES(@v_idEmp,@p_ApeEmp,@p_NomEmp,@p_FecNac,@p_DirEmp,@p_idDist,@p_fonoEmp,@p_idCargo,@p_FecContrata)
SET @P_resultado = 'El empleado con ID ' + trim(str(@v_idEmp))+ ' se insertó correctamente'
END TRY
BEGIN CATCH
SET @P_resultado = 'Ocurrió un error con el insert ' +ERROR_PROCEDURE() + ' -- ' + ERROR_MESSAGE()
END CATCH

END
ELSE
SET @P_resultado = 'El distrito con ID ' + trim(str(@p_idDist))+ ' no existe'

End
```

PROCEDIMIENTOS ALMACENADOS



```
CREATE PROCEDURE Compras.uspInsertaEmpleado
    @p_ApeEmp varchar(50),
    @p_NomEmp varchar(50),
    @p_FecNac datetime,
    @p_DirEmp varchar(60),
    @p_idDist int,
    @p_fonoEmp varchar(15),
    @p_idCargo int,
    @p_FecContrata datetime,
    @P_resultado varchar(500) OUTPUT
AS
Begin
    declare @v_idEmp int

    --Validamos la existencia del idDistrito porintegridad referencial
    IF EXISTS(SELECT * FROM RRHH.Distritos WHERE idDistrito = @p_idDist)
    BEGIN
        --Calculamos el nuevo Id de empleado
        SELECT @v_idEmp = MAX(IdEmpleado) + 1 FROM RRHH.empleados
        --controlamos el error en la inserción
        BEGIN TRY
            INSERT INTO RRHH.empleados VALUES(@v_idEmp,@p_ApeEmp,@p_NomEmp,@p_FecNac,@p_DirEmp,@p_idDist,@p_fonoEmp,@p_idCargo,@p_FecContrata)
            SET @P_resultado = 'El empleado con ID ' + trim(str(@v_idEmp))+ ' se insertó correctamente'
        END TRY
        BEGIN CATCH
            SET @P_resultado = 'Ocurrió un error con el insert ' +ERROR_PROCEDURE() + ' -- ' + ERROR_MESSAGE()
        END CATCH
    END
    ELSE
        SET @P_resultado = 'El distrito con ID ' + trim(str(@p_idDist))+ ' no existe'

End
```

PROCEDIMIENTOS ALMACENADOS



```
set dateformat dmy
declare @v_resultado varchar(300)
exec Compras.uspInsertaEmpleado
    @p_ApeEmp = 'Chavez',
    @p_NomEmp = 'Victor',
    @p_FecNac = '18/04/1980',
    @p_DirEmp = 'Su casa',
    @p_idDist = 999,
    @p_fonoEmp = '999999999',
    @p_idCargo = 100,
    @p_FecContrata = '20/03/2023',
    @P_resultado = @v_resultado OUTPUT
print @v_resultado
```

100 %

Messages

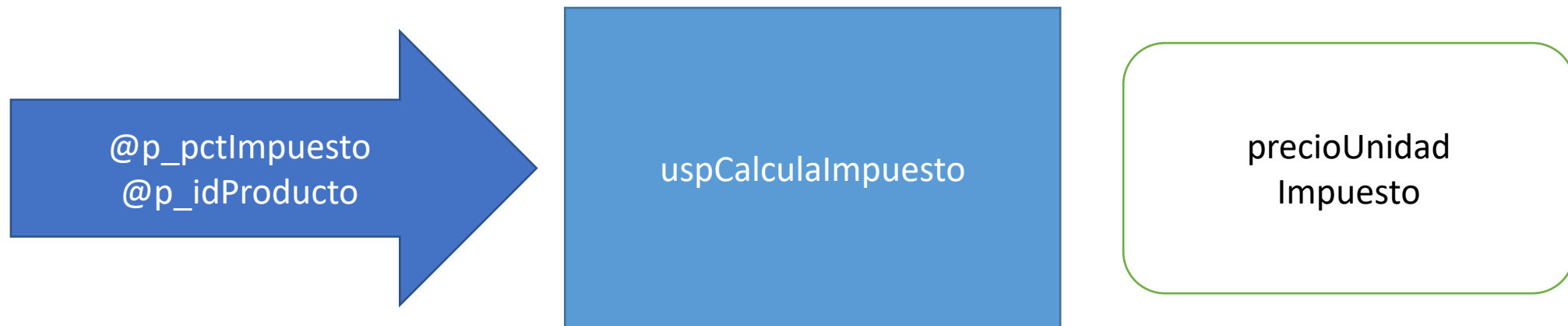
El distrito con ID 999 no existe

Completion time: 2023-02-25T19:17:34.8095909-05:00



INTEGRADOR

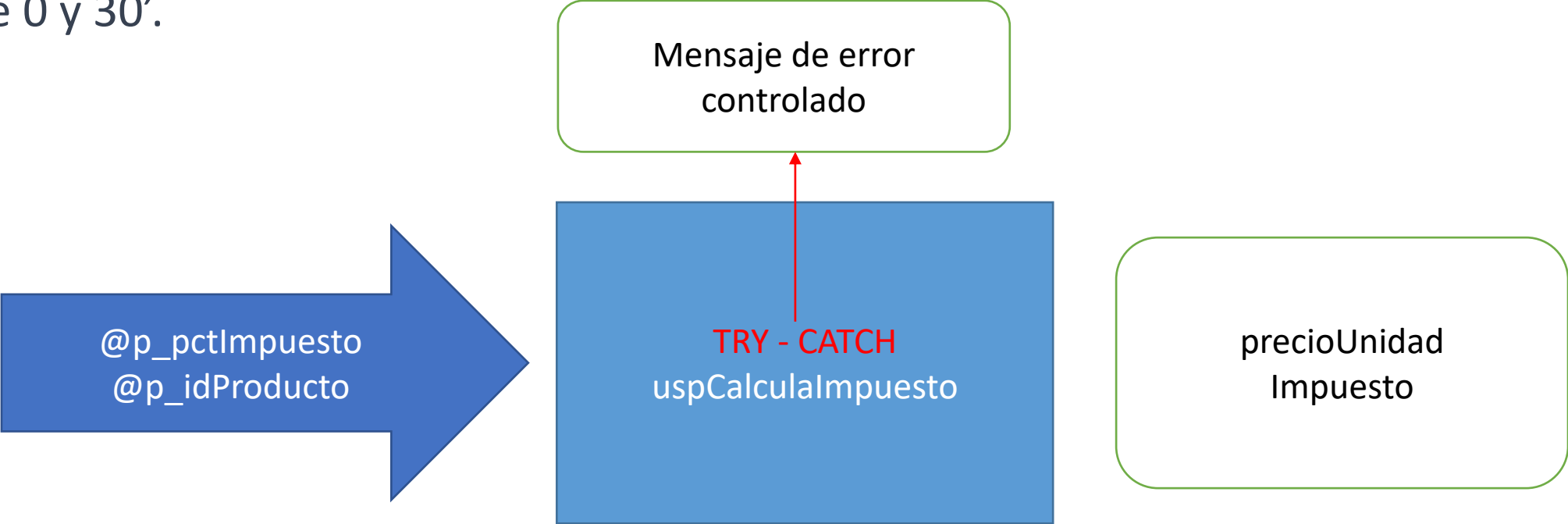
Crear un procedimiento con nombre uspCalculaImpuesto que calcule el monto de impuesto a pagar en base al porcentaje del impuesto y código del producto.



PROCEDIMIENTOS ALMACENADOS



Modifique el procedimiento **uspCalculaImpuesto** para que valide que el porcentaje de impuesto se encuentre entre 0 y 30 caso contrario genere un error de severidad 16 usando RAISERROR que muestre el mensaje 'El porcentaje de impuesto debe estar entre 0 y 30'.





VISTAS

VISTAS



¿Qué es una vista?

- Una vista es una tabla lógica basada en una tabla u otra vista.
- Una vista no contiene datos propios, pero es como una ventana a través de la cual se pueden ver o cambiar los datos de las tablas.
- Las tablas en las que se basa una vista se denominan tablas base.

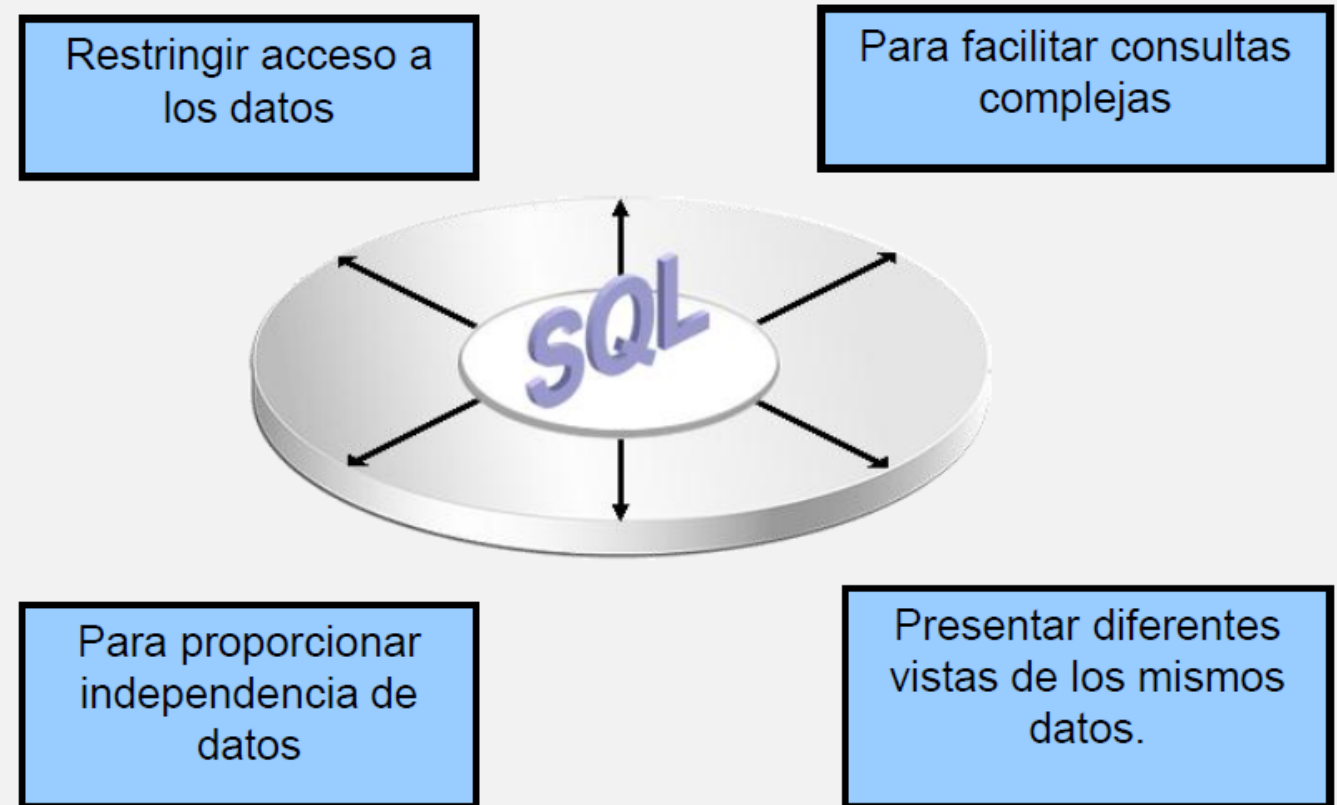
EMPLOYEES table

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY
1	100 Steven	King	SKING	515.123.4567	17-JUN-87	AD_PRES	24000
2	101 Neena	Kochhar	NKOCHH...	515.123.4568	21-SEP-89	AD_VP	17000
3	102 Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-93	AD_VP	17000
4	103 Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-90	IT_PROG	9000
5						IT_PROG	6000
6						IT_PROG	4200
7						IT_PROG	5800
						IT_PROG	3500
						IT_PROG	3100
						IT_PROG	2600
						IT_PROG	2500
						IT_PROG	10500
						IT_PROG	11000
						IT_PROG	8600
						IT_PROG	7000
						IT_PROG	4400
						IT_PROG	13000
						IT_PROG	6000
						IT_PROG	12000
						IT_PROG	8300



Ventajas de las vistas

- Las vistas restringen el acceso a los datos porque muestra las columnas seleccionadas de la tabla.
- Las vistas se pueden utilizar para realizar consultas simples para recuperar los resultados de consultas complicadas.
- Las vistas proporcionan independencia de datos para usuarios ad hoc y programas de aplicación.
- Las vistas proporcionan a los grupos de usuarios acceso a los datos de acuerdo con sus criterios particulares.





Vistas simples y vistas complejas

Características	Vista simple	Vista compleja
Numero de tablas	Uno	Uno o mas
Contiene funciones	No	Si
Contiene grupo de datos	No	Si
Operaciones DML a través de una vista	Si	No siempre



¿Preguntas o comentarios?



CONCLUSIONES

Una excepción se genera ante la ocurrencia de un error durante la ejecución del programa. Esto nos permitirá controlar o reaccionar ante estos errores.

Un procedimiento almacenado en SQL Server es una herramienta útil y eficaz para mejorar el rendimiento y la seguridad de la base de datos, simplificar el código de las aplicaciones y modularizar las instrucciones SQL reutilizables.

La vista es una “Tabla Lógica” que hace referencia a una consulta de una o varias tablas. La vista en si no contiene datos.



REFERENCIAS BIBLIOGRÁFICAS



TRY...CATCH (Transact-SQL)

<https://learn.microsoft.com/es-es/sql/t-sql/language-elements/try-catch-transact-sql?view=sql-server-ver16>

Gravedades de errores del motor de base de datos

<https://learn.microsoft.com/es-es/sql/relational-databases/errors-events/database-engine-error-severities?view=sql-server-ver16>

Raiserror

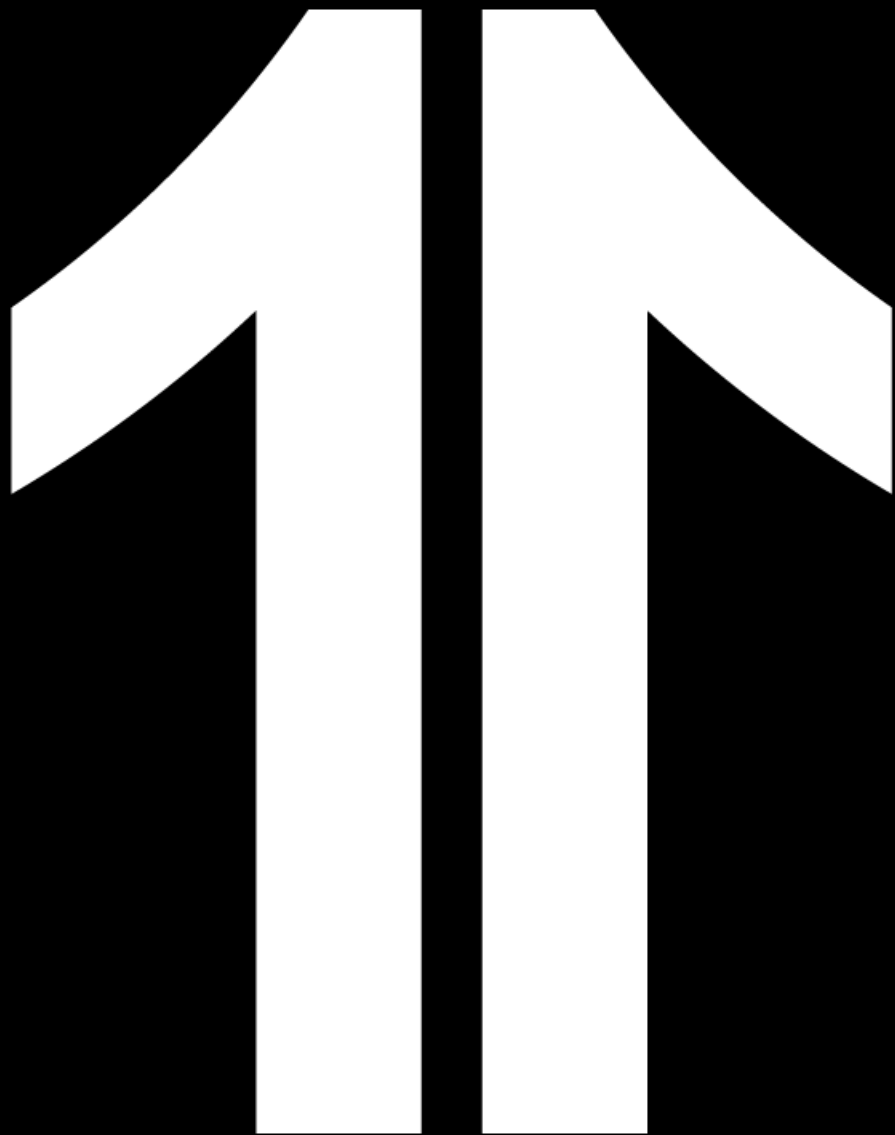
<https://learn.microsoft.com/es-es/sql/t-sql/language-elements/raiserror-transact-sql?view=sql-server-ver16>

CREATE PROCEDURE (Transact-SQL)

<https://learn.microsoft.com/es-es/sql/t-sql/statements/create-procedure-transact-sql?view=sql-server-ver16>

Creación de vistas

<https://learn.microsoft.com/es-es/sql/relational-databases/views/create-views?view=sql-server-ver16>



GRACIAS

